

Examen de Fundamentos de Ingeniería del Software

Publicación de notas: Lunes, 18 de mayo de 2026

Revisión: Miércoles , 20 de mayo de 2026, 13:00

Es preciso identificarse mediante apellidos y nombre, así como el carné de la UPM (o en su defecto DNI o pasaporte). Cualquier dispositivo electrónico, incluidos teléfonos móviles, debe estar apagado y guardado. Es obligatorio entregar el examen, aunque no conteste a ninguna pregunta. No se puede salir del examen durante los primeros 20 minutos. Este examen tiene una duración de 60 minutos y consta de 15 preguntas. Existen diferentes versiones del examen.

Cada pregunta solo tiene una respuesta correcta y todas las preguntas tienen la misma puntuación. Cada respuesta correcta suma 1 punto y cada respuesta incorrecta resta 0,33 puntos. Las respuestas en blanco no suman ni restan.

La RESPUESTA DEFINITIVA SE DEBE RELLENAR EN LA PLANTILLA DE RESPUESTAS. Solo se tendrán en cuenta las respuestas especificadas en la plantilla, no se tendrán en cuenta las anotaciones o contestaciones en el resto del examen.

Ejemplos de relleno de DNI/NIE/PASSPORT

	7	0	9	8	7	2	4	0	7	4	2	5	2	3	4	8	0	0
0																		
1																		
2																		
3																		
4																		
5																		
6																		
7																		
8																		
9																		

	ES	4	2	5	2	3	4	8	0	0	4	4	2	5	2	3	4	8	0
0																			
1																			
2																			
3																			
4																			
5																			
6																			
7																			
8																			
9																			

Ejemplos de Respuesta

	4	4	2	5	2	3	4	8	0
0									
1									
2									
3									
4									
5									
6									
7									
8									
9									



Pregunta 1:

Durante la fase de diseño de una aplicación Android para la consulta de información sobre películas, se ha añadido una nueva clase al diagrama de clases cuya responsabilidad es almacenar las listas de películas favoritas del usuario utilizando una base de datos local. ¿Qué tipo de clase es la que se ha añadido al diagrama?

- (a) Clase de interfaz de usuario.
- (b) Controlador.
- (c) Clase del dominio.
- (d) **Clase de persistencia.**

Pregunta 2:

Indique con cuál de los siguientes términos encaja mejor la siguiente definición: “Elemento definido por UML que representa un recurso computacional capaz de ejecutar y alojar artefactos de un sistema software”.

- (a) Componente.
- (b) Paquete.
- (c) **Nodo.**
- (d) Servidor.

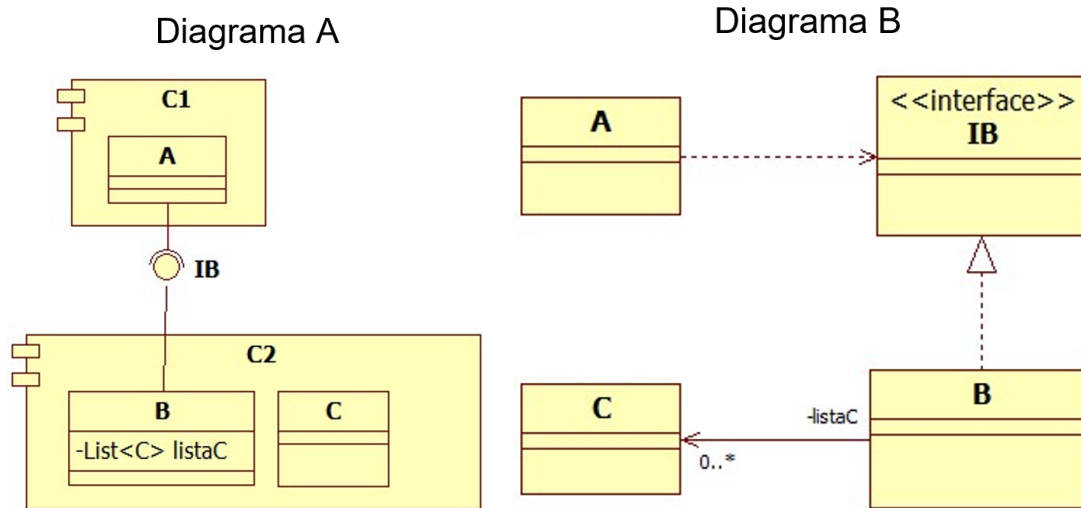
Pregunta 3:

En un sistema software que sigue el patrón Modelo Vista Controlador (MVC), se requiere recuperar un objeto complejo del Modelo que contiene datos sensibles y estructuras de datos crudas. Antes de mostrar la información al usuario, el sistema debe filtrar los campos privados y mapear los datos a un formato específico requerido por la interfaz de usuario. ¿Qué componente es el responsable de realizar esta transformación de los datos?

- (a) El Modelo, ya que es el componente que gestiona los datos y encapsula la información de la aplicación.
- (b) La Vista, ya que es el componente que encapsula qué subconjunto de información se muestra y cómo se presenta al usuario.
- (c) **El Controlador, ya que entre sus funciones está la de preparar los datos para la vista.**
- (d) El patrón MVC no permite la transformación de los datos, deben fluir directamente del Modelo a la Vista.

Pregunta 4:

Los diagramas A y B representan el diseño de un software desde dos perspectivas diferentes. ¿Es coherente el diagrama A con el diagrama B?



- (a) Sí.
- (b) No, porque según el diagrama A la clase A implementa la interfaz IB y según el diagrama B la clase A usa la interfaz IB.
- (c) **No, porque el diagrama A indica que la clase B tiene como atributo una lista de objetos de la clase C, algo que no se refleja adecuadamente en el diagrama B.**
- (d) No, porque el diagrama B marca claramente que el elemento IB es una interfaz y el diagrama A no.

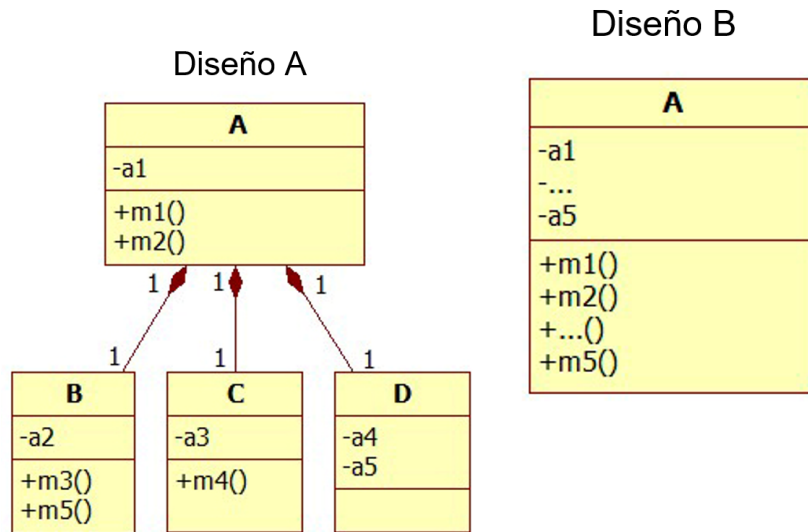
Pregunta 5:

Se pretenden realizar pruebas de caja negra utilizando la técnica de análisis de valores límite para probar una función que recibe como único parámetro un número entero cuyos valores mínimo y máximo válidos son 0 y 100, respectivamente. ¿Qué conjunto mínimo de casos de prueba se debería definir de acuerdo a esta técnica?

- (a) CP1 (0), CP2 (100).
- (b) CP1 (-1), CP2 (50), CP3 (101).
- (c) **CP1 (-1), CP2 (0), CP3 (100), CP4 (101).**
- (d) CP1 (-1), CP2 (0), CP3 (50), CP4 (100), CP5 (101).

Pregunta 6:

Para pasar del diseño A (mostrado a la izquierda) al diseño B (mostrado a la derecha), se ha aplicado:



- (a) Abstracción por generalización.
- (b) Abstracción por agrupación.
- (c) **Abstracción por agregación.**
- (d) Abstracción por descomposición.

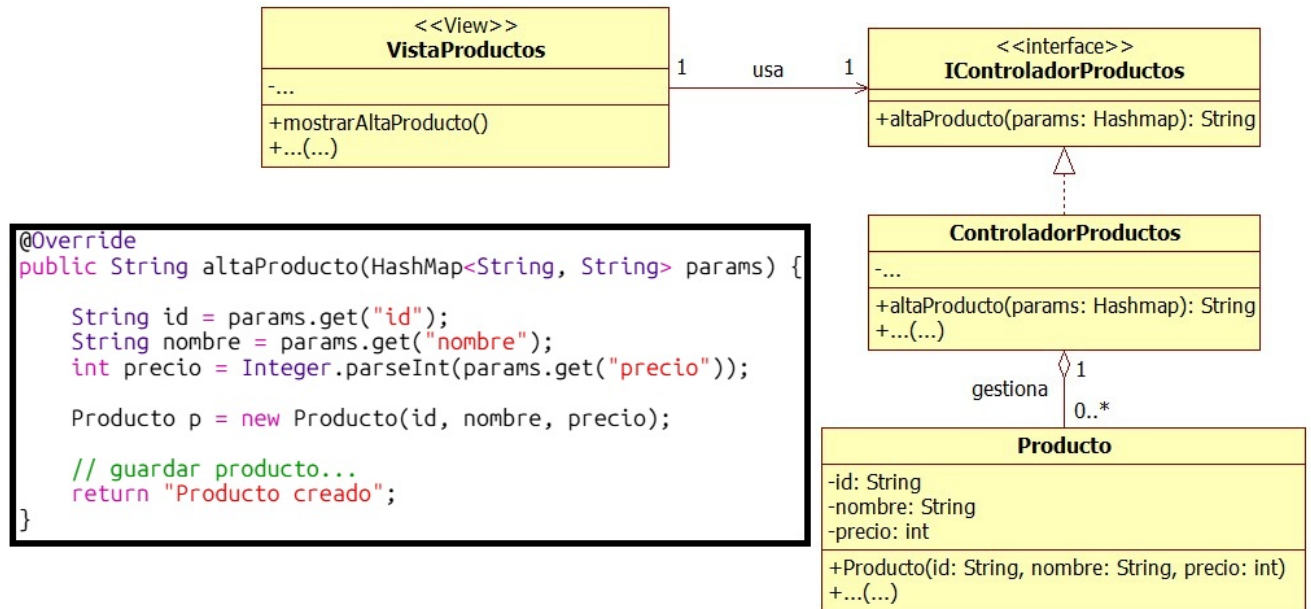
Pregunta 7:

Un ingeniero de software quiere realizar pruebas de caja negra utilizando la técnica de particiones de equivalencia para probar un método Java que recibe dos parámetros: una cadena de caracteres que no debe ser nula y cuya longitud debe estar entre 1 y 5, y un número que debe ser mayor de 0 y menor de 100. Este ingeniero ha definido los siguientes casos de prueba: CP1 ("ABC",0), CP2 (null,1), CP3 ("",1) y CP4 ("ABC",100). ¿Cuántas clases de equivalencia faltan por probar de forma adecuada?

- (a) 4.
- (b) **3.**
- (c) 2.
- (d) 1.

Pregunta 8:

Dado el siguiente diagrama de clases y considerando que el método `altaProducto` permite crear instancias de la clase `Producto`, es posible afirmar que el acoplamiento existente entre la clase `VistaProductos` y la clase `ControladorProductos` es:



- (a) El mínimo acoplamiento posible.
- (b) **Un acoplamiento de marcado.**
- (c) Un acoplamiento de datos.
- (d) Un acoplamiento de contenido.

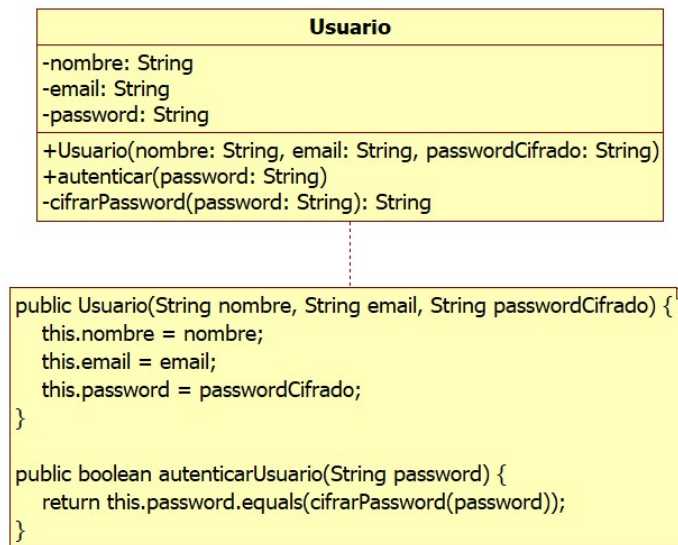
Pregunta 9:

¿Cuál de las siguientes diferencias entre las pruebas de caja blanca y de caja negra es VERDADERA?

- (a) **Las pruebas de caja blanca requieren conocer la implementación del software mientras que las pruebas de caja negra pueden ser realizadas conociendo únicamente la definición de la interfaz del software.**
- (b) Las pruebas de caja blanca permiten garantizar que un software está libre de errores, mientras que esto no es posible empleando pruebas de caja negra.
- (c) Generalmente, las pruebas de caja blanca se diseñan para probar requisitos funcionales, mientras que las pruebas de caja negra se diseñan para probar requisitos no funcionales.
- (d) A diferencia de las pruebas de caja negra, las pruebas de caja blanca son siempre pruebas unitarias.

Pregunta 10:

Dado el siguiente diagrama de clases, se puede afirmar que el diseño:



- (a) Cumple con el principio de responsabilidad única.
- (b) No cumple con el principio de responsabilidad única porque el constructor de la clase **Usuario** debe conocer el algoritmo de cifrado de las contraseñas.
- (c) **No cumple con el principio de responsabilidad única porque la clase **Usuario** incluye la lógica de autenticación.**
- (d) No cumple con el principio de responsabilidad única porque la clase **Usuario** se encarga de la presentación de la información relativa a los usuarios.

Pregunta 11:

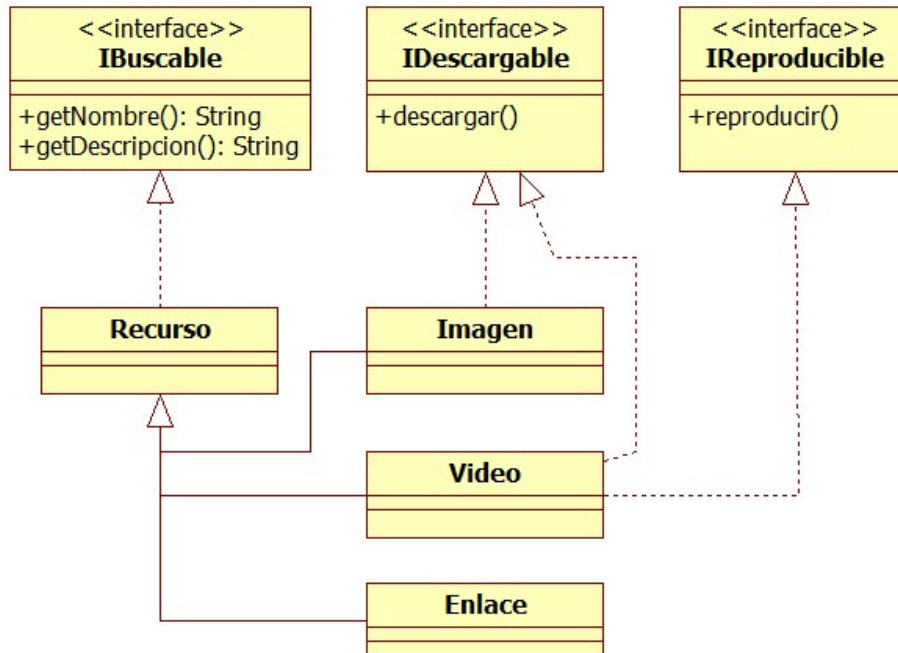
De acuerdo al diagrama de clases adjunto, se puede afirmar que:



- (a) **La clase A tiene como atributo un objeto de una clase que implementa la interfaz IB.**
- (b) La clase A usa un objeto que implementa la interfaz IB, pero no tiene como atributo ningún objeto que implemente dicha interfaz.
- (c) La clase A implementa la interfaz IB.
- (d) La clase A tiene como atributo un objeto del que necesita conocer su clase concreta para invocar cualquiera de sus métodos.

Pregunta 12:

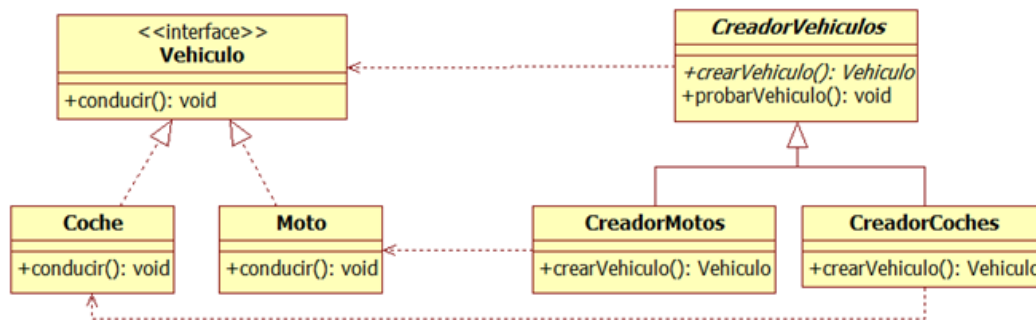
Dado el siguiente diagrama de clases y considerando que se ha aplicado el principio de segregación de interfaces, se puede afirmar que:



- (a) No se ha aplicado correctamente el principio de segregación de interfaces porque la interfaz **IBuscable** debería dividirse en dos interfaces más específicas.
- (b) No se ha aplicado correctamente el principio de segregación de interfaces porque las clases **Imagen** y **Video** no proporcionan la interfaz **IBuscable**, siendo las imágenes y los vídeos recursos que pueden ser buscados.
- (c) No se ha aplicado correctamente el principio de segregación de interfaces porque las clases deberían depender de una única interfaz.
- (d) **Ninguna de las otras respuestas es verdadera.**

Pregunta 13:

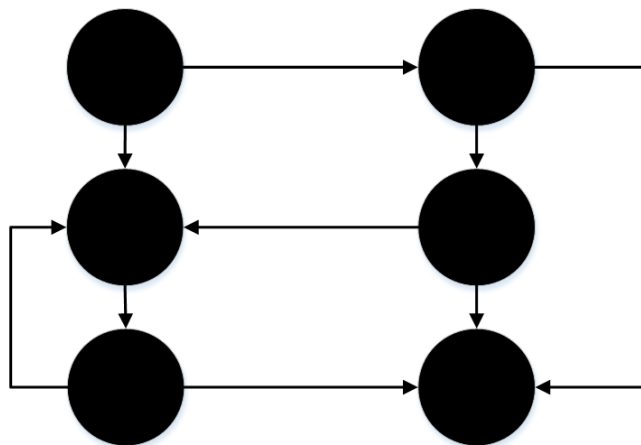
¿Qué patrón de diseño se ha aplicado en el siguiente diagrama de clases?



- (a) Proxy.
- (b) **Factory Method.**
- (c) Command.
- (d) Ninguno.

Pregunta 14:

Dado el siguiente grafo de flujo construido mediante la prueba de la ruta base, ¿cuántos casos de prueba serán necesarios para cubrir todas las rutas independientes del conjunto base?

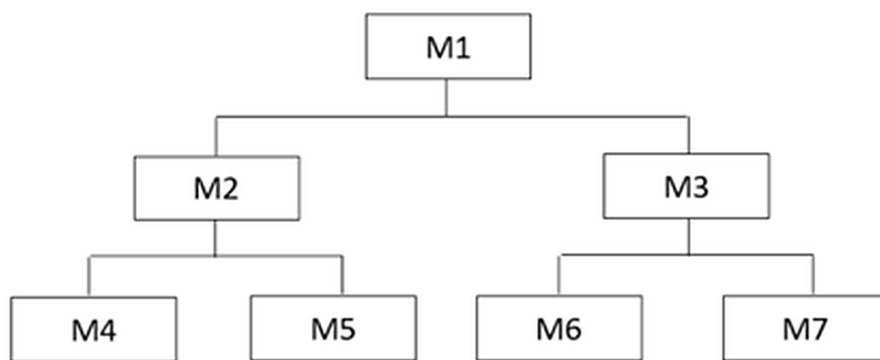


- (a) 6.
- (b) **5.**
- (c) 4.
- (d) 3.

Pregunta 15:

Se pretenden realizar las pruebas de integración de un sistema software mediante una estrategia de integración ascendente. El sistema tiene los módulos mostrados en la figura adjunta, los cuales están organizados jerárquicamente. Analice las siguientes afirmaciones y señale la respuesta verdadera.

- I. Para probar los módulos intermedios (M2 y M3), es necesario desarrollar un controlador o *driver* que sustituya la funcionalidad de M1.
- II. Para probar los módulos del nivel intermedio (M2 y M3) se deben haber probado antes los del nivel inferior (M4, M5, M6 y M7).
- III. En ningún momento va a resultar necesario crear esqueletos o *stubs*.



- (a) Solo I y II son verdaderas.
- (b) Solo II y III son verdaderas.
- (c) Solo I y III son verdaderas.
- (d) **I, II y III son verdaderas.**